CO1 Assessment 1: Analytical Problem Solving

QUESTION 1: Bridge and Torch Problem (State-Space + Uniform Cost Search)

1.1 Definition

Four people must cross a bridge at night. The bridge holds at most 2 people at a time, and they need the single torch. A pair walks at the speed of the slower person. The goal is to get everyone across in the minimum total time. It is a classic state-space search problem solved with Uniform Cost Search (UCS).

Crossing times: A = 1, B = 2, C = 5, D = 10 minutes.

1.2 Problem Formulation

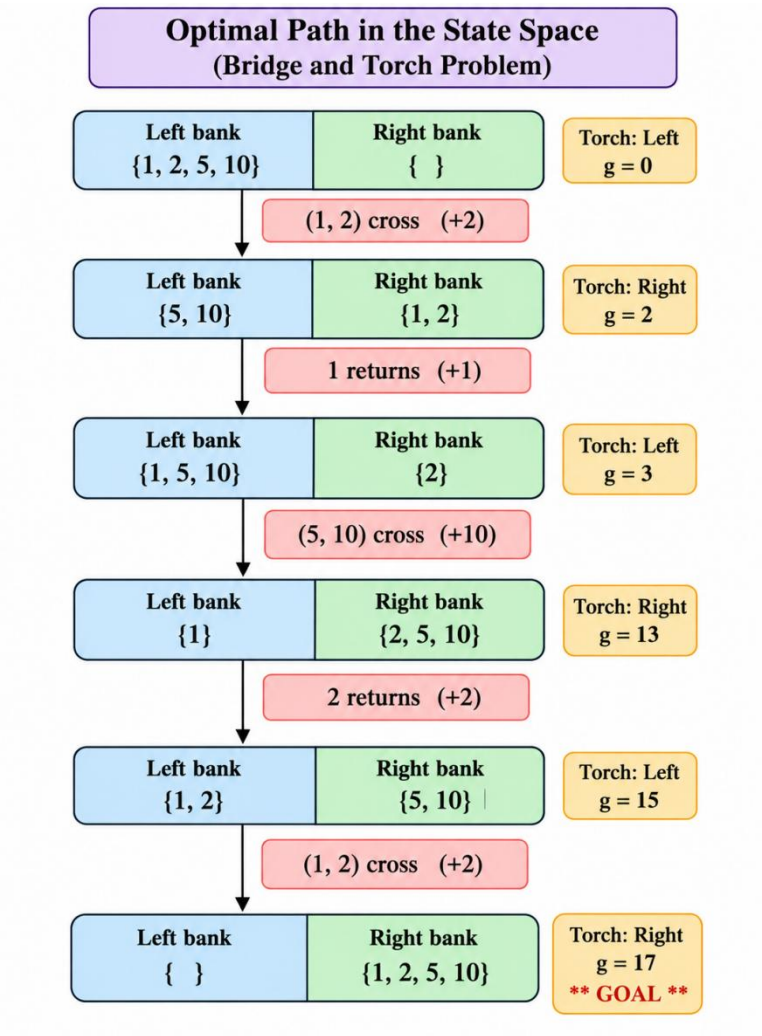
Component	Description
State	(people on Left bank, people on Right bank, torch side)
Initial state	({1,2,5,10}, { }, Torch = Left)
Goal state	({ }, {1,2,5,10}, Torch = Right)
Actions	Move 1 or 2 people (on the torch side) across with the torch
Path cost	Time of a crossing = **max** of the walking times of those crossing; total cost = sum
Goal test	Left bank is empty
State space size	Each person is Left/Right ($2^4 = 16$) × torch side (2) = **at most 32 states**

1.3 Algorithm (UCS)

1. Put the initial state in the **frontier** (priority queue ordered by g) with g = 0.
2. If the frontier is empty, report failure.
3. Remove the node with the **lowest g** from the frontier.

4. If it passes the goal test, return the solution.
5. Add it to the explored set.
6. Generate all successors. For each one, g = g(parent) + crossing time.
7. Insert a successor if it is new, or replace the old entry if it has a lower g. Ignore it if it is already explored.
8. Go to step 2.

1.4 Diagram: Optimal Path in the State Space



1.5 Example and Step-by-Step Solution

(a) UCS expansion trace (nodes popped in order of g)

Step	Node expanded (g)	Successors generated (g)
1	[1,2,5,10 | –] L (0)	[5,10|1,2]R **2**; [2,10|1,5]R 5; [1,10|2,5]R 5; [2,5|1,10]R 10; [1,5|2,10]R 10; [1,2|5,10]R 10
2	[5,10|1,2] R (2)	1 returns → [1,5,10|2]L **3**; 2 returns → [2,5,10|1]L 4
3	[1,5,10|2] L (3)	(1,5) → [10|1,2,5]R 8; (1,10) → [5|1,2,10]R 13; (5,10) → [1|2,5,10]R **13**
4	[2,5,10|1] L (4)	(2,5) → [10|1,2,5]R 9 ✗ (8 already better); (2,10) → [5|1,2,10]R 14 ✗; (5,10) → [2|1,5,10]R 14
5	[2,10|1,5] R (5)	1 returns → [1,2,10|5]L 6; 5 returns → worse, discarded
6	[1,10|2,5] R (5)	2 returns → [1,2,10|5]L 7 ✗ (6 already better)
7	[1,2,10|5] L (6)	(1,2) → [10|1,2,5]R 8 (tie, no change); others cost 16 ✗
8	[10|1,2,5] R (8)	1 returns → [1,10|2,5]L 9; 2 returns → [2,10|1,5]L 10; 5 returns → [5,10|1,2]L 13
9	[1,10|2,5] L (9)	(1,10) crosses → goal found with g = **19** (not yet popped)
10	States with g = 10 to 14	Only cheaper route found: [1,2,5|10]L (11) → [5|1,2,10]R (13); [1,5|2,10]L (14) → goal 19. None beats 17
11	[1,2|5,10] L (**15**)	(1,2) crosses → **Goal with g = 17**
12	Goal (17) popped	**UCS stops. Optimal cost = 17**

(b) Crossing sequence and total time

Trip	Who moves	Direction	Time	Running total
1	1 and 2	Left → Right	max(1,2) = 2	2
2	1 (with torch)	Right → Left	1	3
3	5 and 10	Left → Right	max(5,10) = 10	13
4	2 (with torch)	Right → Left	2	15
5	1 and 2	Left → Right	max(1,2) = 2	**17**

Minimum total time = 2 + 1 + 10 + 2 + 2 = 17 minutes.

(c) Comparison with the obvious strategy

Strategy	Calculation	Total
Fastest person escorts everyone	2 + 1 + 5 + 1 + 10	19
Optimal (the two slowest cross together)	2 + 1 + 10 + 2 + 2	**17**

The two slowest people (5 and 10) cross together, so the 10-minute cost is paid only once.

1.6 Pseudocode

```
function UCS(problem):
   node ← (state = problem.INITIAL, g = 0, parent = NULL)
   frontier ← priority queue ordered by g, containing node
   explored ← empty set
   loop:
      if frontier is empty: return FAILURE
      node ← frontier.POP_LOWEST_g()
      if problem.GOAL_TEST(node.state): return SOLUTION(node)
      add node.state to explored
      for each action in problem.ACTIONS(node.state):
```

```
child.state ← RESULT(node.state, action)
child.g ← node.g + STEP_COST(node.state, action)
if child.state not in explored and not in frontier:
    frontier.INSERT(child)
else if child.state in frontier with higher g:
    replace that entry with child
```

1.7 Conclusion

The Bridge and Torch problem is modelled as a state-space search with the torch side included in the state and the crossing time as the action cost. UCS expands the cheapest state first and finds the optimal plan (1,2 → 1 back → 5,10 → 2 back → 1,2) with a minimum total time of **17 minutes**.

QUESTION 2: Route Finding using A* Search

2.1 Definition

A* is an informed search algorithm. It expands the node with the lowest

f(n) = g(n) + h(n)

where g(n) is the actual cost so far and h(n) is a heuristic estimate of the remaining cost. It combines UCS (uses g) and greedy search (uses h).

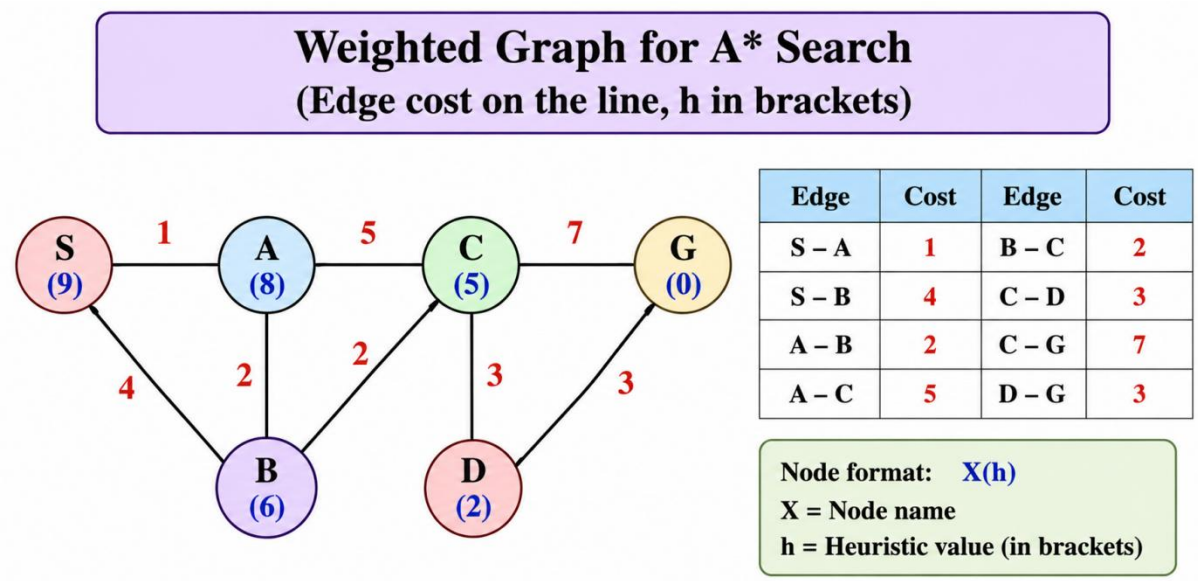
2.2 Problem Formulation

Component	Description
State	A node (city/junction)
Initial state	S
Goal state	G
Actions	Travel along a road (edge) to a neighbouring node
State space	The graph's nodes: {S, A, B, C, D, G}

Path cost	Sum of edge costs
Goal test	n = G

2.3 Algorithm

1. Put the start node in OPEN with g = 0 and f = h(start).
2. Repeat while OPEN is not empty.
3. Remove from OPEN the node n with the **lowest f(n)**.
4. If n is the goal, stop and return the path (follow parent links).
5. Move n to CLOSED.
6. For each neighbour m of n, compute g(m) = g(n) + cost(n,m) and f(m) = g(m) + h(m).
7. If m is in CLOSED, skip it. If m is in OPEN with a larger g, **update** its g, f and parent. If m is new, add it to OPEN.
8. Go to step 2.



Node	**S**	**A**	**B**	**C**	**D**	**G**
h(n)	9	8	6	5	2	0

Check of h (true remaining cost h*): S = 11, A = 10, B = 8, C = 6, D = 3, G = 0. Every h(n) ≤ h*(n), so h is **admissible**.

2.5 Step-by-Step Solution

Step	Expanded node	Neighbours: g, h, f	OPEN after step (f)	CLOSED
1	**S** (f = 9)	A: g = 1, h = 8, **f = 9**; B: g = 4, h = 6, f = 10	A(9), B(10)	S
2	**A** (f = 9)	B: g = 1+2 = 3, h = 6, **f = 9** (better than 4, **update**); C: g = 1+5 = 6, h = 5, f = 11	B(9), C(11)	S, A
3	**B** (f = 9)	C: g = 3+2 = 5, h = 5, **f = 10** (better than 6, **update**)	C(10)	S, A, B
4	**C** (f = 10)	D: g = 5+3 = 8, h = 2, **f = 10**; G: g = 5+7 = 12, h = 0, f = 12	D(10), G(12)	S, A, B, C
5	**D** (f = 10)	G: g = 8+3 = 11, h = 0, **f = 11** (better than 12, **update**)	G(11)	S, A, B, C, D
6	**G** (f = 11)	Goal reached	–	–

Summary table of final values

Node	g(n)	h(n)	f(n) = g + h	Best parent
S	0	9	9	–
A	1	8	9	S
B	3	6	9	A
C	5	5	10	B
D	8	2	10	C

G	11	0	11	D

Expansion order: S → A → B → C → D → G
Final path: S → A → B → C → D → G, cost = 1 + 2 + 2 + 3 + 3 = 11

Other paths are costlier: S-A-C-D-G = 12, S-B-C-D-G = 12, S-A-C-G = 13.

2.6 Pseudocode

```
function A_STAR(start, goal):
   OPEN ← priority queue ordered by f; insert start with g = 0, f = h(start)
   CLOSED ← empty set
   while OPEN is not empty:
      n ← OPEN.POP_LOWEST_f()
      if n = goal: return PATH(n)
      add n to CLOSED
      for each neighbour m of n with edge cost c(n,m):
         g_new ← g(n) + c(n,m)
         if m in CLOSED: continue
         if m not in OPEN or g_new < g(m):
            g(m) ← g_new
            f(m) ← g(m) + h(m)
            parent(m) ← n
            OPEN.INSERT_OR_UPDATE(m)
   return FAILURE
```

2.7 When Is A* Optimal?

A* guarantees an optimal solution when:

1. **h is admissible**: h(n) ≤ true cost to the goal (it never overestimates). This is enough for tree search.
2. For graph search with a CLOSED list, **h should be consistent**: h(n) ≤ c(n,n′) + h(n′).
3. All edge costs are **positive**, and the branching factor is finite.

In our graph h is consistent. For example, h(A) = 8 ≤ cost(A,B) + h(B) = 2 + 6 = 8. Special case: **h = 0 makes A* identical to UCS.**

2.8 Advantages and Limitations

Advantages	Limitations

Optimal if h is admissible/consistent	Quality depends on a good heuristic
Complete on finite graphs with positive costs	Stores all generated nodes, so **memory heavy**
Expands fewer nodes than UCS (guided by h)	Time can still be exponential in the worst case
Widely used (maps, games, robots)	A bad (overestimating) h can lose optimality

2.9 Conclusion

A* ranks nodes by f(n) = g(n) + h(n). In the example it expanded S, A, B, C, D, G and returned the optimal route S-A-B-C-D-G with cost **11**. Optimality is guaranteed when the heuristic is admissible (and consistent for graph search).

QUESTION 3: Smart Traffic Signal Controller (Intelligent Agent)

3.1 Definition

An **intelligent agent** perceives its environment through **sensors** and acts on it through **actuators** to achieve goals. A **Smart Traffic Signal Controller** is an agent that senses vehicle queues at a junction and changes the signals to **reduce waiting time** while staying **safe**.

3.2 Problem Formulation (PEAS Description)

Assumed junction: a 4-way crossing with approaches **N, S, E, W** and two phases: **NS-green** and **EW-green**.

PEAS	Description
Percepts	Queue length on each approach, waiting time, current phase and timer, emergency vehicle signal, pedestrian button press
Sensors	Inductive loop detectors, cameras, pedestrian push-buttons, emergency-vehicle receiver, timer/clock

Environment	Road junction with cars, buses, two-wheelers, pedestrians, emergency vehicles, and changing traffic and weather
Actions	Keep the current green, extend green, switch phase, show yellow, all-red clearance, give pedestrian walk, give emergency priority
Actuators	Red/yellow/green lamps, pedestrian signals, phase timer relays
Agent program	Rule/utility-based decision logic (see pseudocode)
Architecture	Controller hardware + sensors + actuators + software (internal state, decision module, safety checker)
Performance measures	Average waiting time ↓, queue length ↓, throughput ↑, number of stops ↓, **zero conflicting greens**, emergency response time ↓, fairness (no approach waits too long)

Environment properties: partially observable, stochastic (random arrivals), dynamic, sequential, continuous, with multiple agents (drivers and pedestrians).

Agent type: a **model-based, utility-based agent**. It keeps internal state (queues, phase timer) and picks the action that maximizes the utility "least delay, with safety".

Safety constraints (assumed example values):

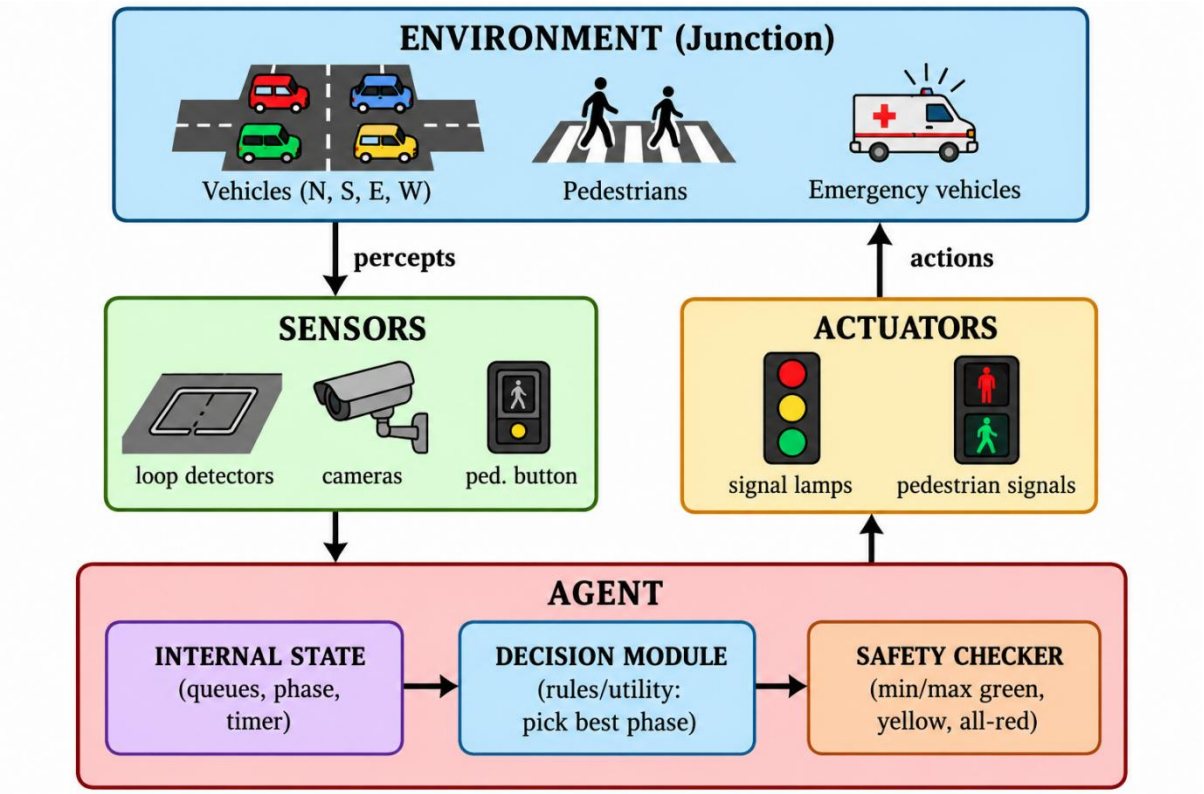
Constraint	Value
Minimum green	10 s
Maximum green	60 s
Yellow	3 s
All-red clearance	2 s
Conflicting greens	**never allowed**

3.3 Algorithm (One Control Cycle)

1. **Sense:** read queue lengths N, S, E, W, pedestrian requests and emergency signals.

2. **Update state:** calculate demand for each phase: NS demand = N + S; EW demand = E + W.
3. **Emergency check:** if an emergency vehicle is detected, give its approach green (after yellow and all-red).
4. **Min-green check:** if the current green has run less than the minimum green, keep it.
5. **Decide:** if the other phase's demand ≥ 2 × the current phase's demand (or the max green has been reached), choose to **switch**.
6. **Safety transition:** show yellow (3 s), then all-red (2 s), then the new green.
7. **Green time:** green = 2 s × (largest queue on the new phase), limited to 10–60 s (assumed ≈ 2 s per vehicle).
8. **Act:** drive the lamps and repeat from step 1.

3.4 Diagram: Agent Architecture



3.5 Example with Different Queue Lengths

Given: N = 12, S = 8, E = 3, W = 2 vehicles. The current phase is **EW-green**, already running for 15 s (above the 10 s minimum).

Quantity	Calculation	Value

NS demand	12 + 8	**20**
EW demand	3 + 2	**5**
Switch rule	20 ≥ 2 × 5 = 10 ✓ and min green is done	**Switch to NS**
Safety sequence	EW yellow 3 s + all-red 2 s	5 s
NS green time	2 s × 12 (largest queue, N) = 24 s, within 10–60	**24 s**
Next EW green	2 s × 3 (E) = 6 s, raised to the minimum	**10 s**

Adaptive cycle length = (24 + 3 + 2) + (10 + 3 + 2) = **44 s**
Fixed-time cycle (30 s green each) = 2 × (30 + 3 + 2) = **70 s**

In the fixed plan, EW would hold green for 30 s although 3 cars need only about 6 s. That wastes about 24 s while 20 vehicles wait on N and S.

3.6 Pseudocode

```
function TRAFFIC_AGENT(percept):
   state ← UPDATE_STATE(state, percept)       // queues, phase, timers
   if state.in_transition:                    // yellow / all-red running
      return CONTINUE_TRANSITION()
   if percept.emergency on approach X:
      return PRIORITY_PHASE(X)                // after yellow + all-red
   if state.green_elapsed < MIN_GREEN:
      return KEEP_CURRENT()
   demand_cur   ← queue sum of current phase
   demand_other ← queue sum of other phase
   if state.green_elapsed >= MAX_GREEN or demand_other >= 2 * demand_cur:
      return START_YELLOW_THEN_SWITCH()
   if percept.pedestrian_request and safe:
      return PEDESTRIAN_PHASE()
   return KEEP_CURRENT()

// when a new green starts:
green_time ← CLAMP(2 * max_queue_of_new_phase, MIN_GREEN, MAX_GREEN)
```

3.7 How It Improves Flow While Staying Safe

- It gives **longer green to heavy queues** and shorter green to light ones, so less time is wasted.
- It **never turns two conflicting directions green together**. The yellow and all-red intervals are fixed and cannot be skipped.
- **Minimum green** stops unsafe flickering. **Maximum green** stops starvation, so every approach gets service.
- Emergency and pedestrian requests are handled with priority, but only through the safe transition sequence.

3.8 Advantages and Limitations

Advantages	Limitations
Adapts to real-time traffic and cuts waiting time	Needs sensors, which can fail or give noisy data
Reduces fuel use and emissions (fewer stops)	Costly to install and maintain
Handles emergency vehicles and pedestrians	Rule thresholds (e.g. 2×) must be tuned
Safety rules are built in	Local view only, unless junctions are coordinated

3.9 Conclusion

The smart traffic controller is a model-based, utility-based agent. It senses queue lengths, updates its internal state, selects the best phase and passes the decision through a safety checker before actuating the lamps. In the example, it switched to NS (demand 20 vs 5) and reduced the cycle from 70 s to 44 s without breaking any safety rule.

QUESTION 4: Problem Formulation: Tower of Hanoi

4.1 Definition

The Tower of Hanoi has **three rods (A, B, C)** and **n disks** of different sizes. All disks start stacked on one rod, with the largest at the bottom. The goal is to move the whole stack to another rod following these rules:

1. Move **only one disk** at a time.
2. Move only the **top disk** of a rod.
3. **Never place a larger disk on a smaller disk.**

4.2 Problem Formulation (n = 3)

Disks are numbered **1 (smallest), 2, 3 (largest)**.

Component	Description
State	Position (A/B/C) of each disk, e.g. (d1, d2, d3). Order on a rod is forced by the size rule
Initial state	All disks on A: (A, A, A), i.e. A = [3,2,1]
Goal state	All disks on C: (C, C, C), i.e. C = [3,2,1]
Legal actions	Move the top disk of one rod to another rod if the target rod is empty or its top disk is larger
State space	$3^n = 3^3 =$ **27 legal states**
Path cost	Each move costs 1, so path cost = number of moves
Goal test	All three disks are on rod C

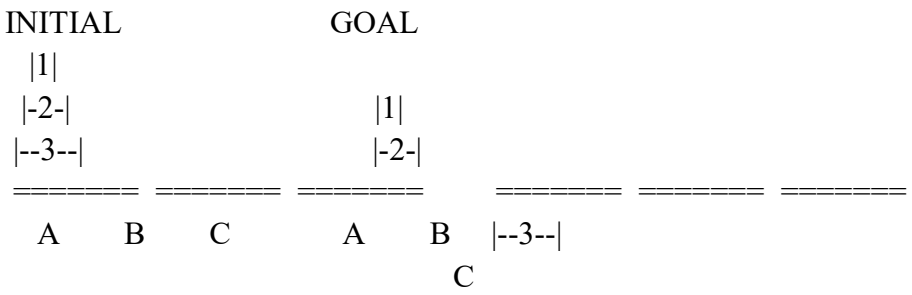
4.3 Algorithm (Recursive)

To move n disks from **Source** to **Target** using **Auxiliary**:

1. If n = 1, move the disk from Source to Target and stop.

2. Move the top **n−1** disks from Source to **Auxiliary** (using Target as helper).
3. Move disk n (the largest) from Source to Target.
4. Move the **n−1** disks from Auxiliary to Target (using Source as helper).

4.4 Diagram: Initial and Goal States, and Recursion Tree



(In the goal state, C holds disks 3 (bottom), 2, 1 (top).)

Recursion tree for hanoi(3, A, C, B):

hanoi(3, A→C, via B)
|-- hanoi(2, A→B, via C)
| |-- hanoi(1, A→C): move disk 1 A→C [Move 1]
| |-- move disk 2 A→B [Move 2]
| |-- hanoi(1, C→B): move disk 1 C→B [Move 3]
|-- move disk 3 A→C [Move 4]
|-- hanoi(2, B→C, via A)
|-- hanoi(1, B→A): move disk 1 B→A [Move 5]
|-- move disk 2 B→C [Move 6]
|-- hanoi(1, A→C): move disk 1 A→C [Move 7]

4.5 Example and Step-by-Step Solution (3 disks, A → C using B)

(Rods are listed bottom → top.)

Move	Action	Rod A	Rod B	Rod C	Cost so far
Start	–	[3,2,1]	[ ]	[ ]	0
1	Disk 1: A → C	[3,2]	[ ]	[1]	1
2	Disk 2: A → B	[3]	[2]	[1]	2

3	Disk 1: C → B	[3]	[2,1]	[ ]	3
4	Disk 3: A → C	[ ]	[2,1]	[3]	4
5	Disk 1: B → A	[1]	[2]	[3]	5
6	Disk 2: B → C	[1]	[ ]	[3,2]	6
7	Disk 1: A → C	[ ]	[ ]	[3,2,1]	**7**

All seven moves are legal. Goal reached with **7 moves**, which is the minimum.

4.6 Pseudocode

```
procedure HANOI(n, source, target, aux):
   if n = 1:
      print "Move disk 1 from", source, "to", target
      return
   HANOI(n-1, source, aux, target)
   print "Move disk", n, "from", source, "to", target
   HANOI(n-1, aux, target, source)

// Call: HANOI(3, A, C, B)
```

4.7 Why $T(n) = 2^n - 1$

Let **T(n)** = number of moves needed for n disks.

- Move n−1 disks away: T(n−1) moves.
- Move the largest disk: **1** move.
- Move n−1 disks back on top: T(n−1) moves.

Recurrence: $T(n) = 2 \cdot T(n-1) + 1$, with $T(1) = 1$

Add 1 to both sides: $T(n) + 1 = 2 \cdot (T(n-1) + 1)$. Repeating gives $T(n) + 1 = 2^{n-1} \cdot (T(1) + 1) = 2^{n-1} \cdot 2 = 2^n$, so

$T(n) = 2^n - 1$

n	**1**	**2**	**3**	**4**	**10**	**20**

T(n) 1 3 **7** 15 1023 1,048,575

Why this is the minimum: the largest disk can move only when the other n−1 disks sit on the third rod. That needs at least T(n−1) moves before and T(n−1) moves after, plus the move itself.

4.8 Advantages and Limitations

Advantages	Limitations
Recursive solution is short and elegant	Moves grow **exponentially** ($2^n - 1$)
Gives the **optimal** number of moves	Infeasible for large n (e.g. 64 disks)
Good model for **problem decomposition**	Recursion uses stack memory (depth n)
State space is small and well-defined (3^n)	Fixed to 3 rods in the basic form

4.9 Conclusion

The Tower of Hanoi is formulated with states as disk positions, initial state (A,A,A), goal state (C,C,C), legal moves of top disks, and unit cost per move. The recursive algorithm solves 3 disks in the 7 moves shown above. The recurrence T(n) = 2T(n−1) + 1 gives **$T(n) = 2^n - 1$**.

QUESTION 5: Mars Rover Agent

5.1 Definition

A **Mars Rover agent** is an **autonomous** robotic agent that senses the Martian surface with sensors and acts with wheels, arms and instruments to **explore safely, save energy, and collect and send scientific data**.

5.2 Problem Formulation (PEAS)

PEAS	Description

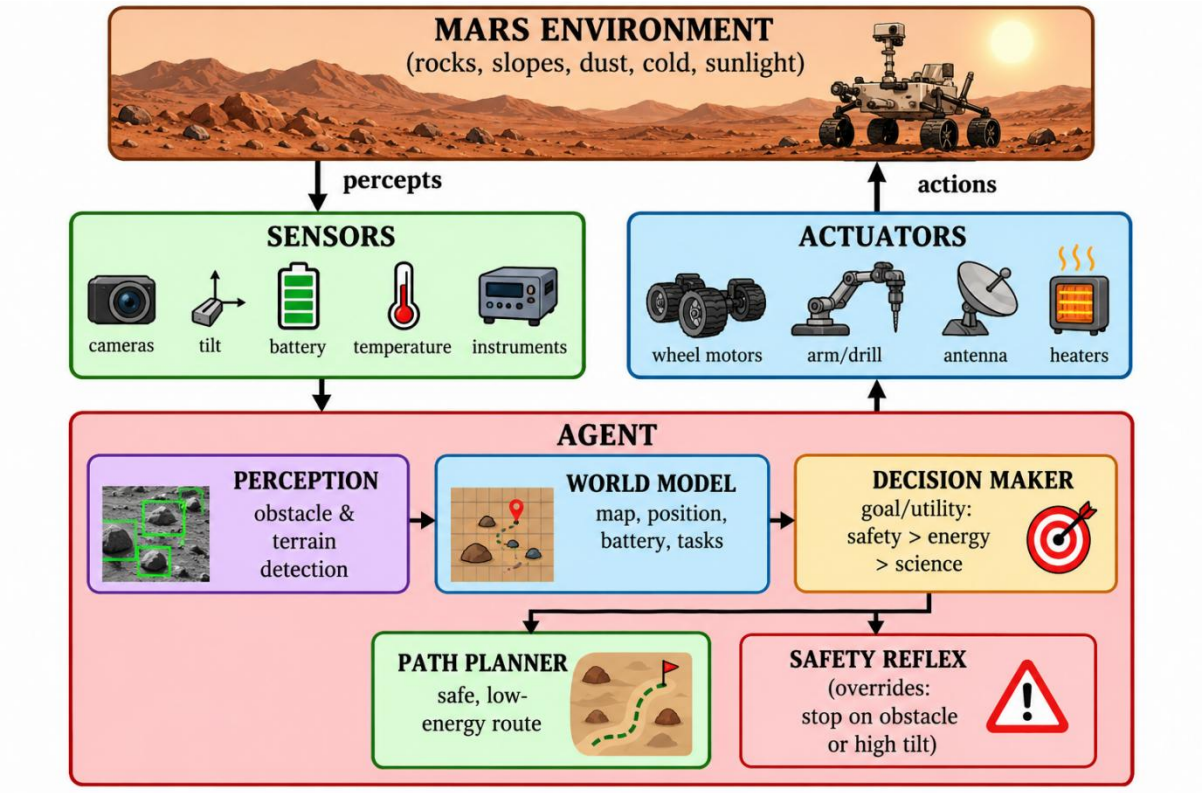
Percepts	Camera images of terrain, obstacle distance, tilt/slope angle, wheel slip, battery level, temperature, sunlight/time of day, sample readings, commands from Earth
Sensors	Stereo cameras, obstacle/range sensor, tilt sensor (IMU), wheel encoders, temperature sensors, battery sensor, sun sensor, science instruments (e.g. spectrometer), radio receiver
Environment	Mars surface: rocks, sand, slopes, craters, dust, low temperature. It is **partially observable, dynamic, stochastic and continuous**, with a long **communication delay** to Earth
Actions	Move forward, turn, stop, reverse, re-plan route, capture image, deploy arm, drill/scoop, analyse sample, charge/sleep, transmit data
Actuators	Wheel and steering motors, robotic arm, drill, camera mast motor, antenna, heaters, transmitter
Performance measure	Science data collected and sent ↑, area explored ↑, **safety (no collision, tip-over or getting stuck)**, **energy efficiency**, mission lifetime ↑

Decision priorities: (1) safety, (2) energy and survival, (3) science goals.

5.3 Algorithm (Rover Control Loop)

1. **Sense:** read cameras, tilt, battery, temperature and the current goal.
2. **Perceive:** build or update a map of obstacles and safe cells.
3. **Safety reflex:** if an obstacle is within the stop distance or the tilt exceeds its limit, **stop** immediately.
4. **Energy check:** if battery < reserve level, go to charge/sleep mode and skip science tasks.
5. **Plan:** find the lowest-cost **safe** path to the target (e.g. A* with obstacles and unsafe slopes blocked).
6. **Act:** move one step along the path, or run the science task if at the target.
7. **Record and transmit:** store data and send it to Earth when the link is available.
8. Repeat from step 1.

5.4 Diagram: Agent Architecture



Earth link: receive commands / send data <--> COMMUNICATION MODULE

5.5 Example: Obstacle Detection and Safe Route

Assumptions (illustration only): the map is a grid of 1 m cells and a move costs 1 energy unit per cell. Row y = −1 is a steep slope (unsafe). The rover starts at **R(0,0)** and the sample site is **G(4,0)**.

```
y=+1          *-----*-----*
              |           |
 y= 0    R-----*    [X]    *-----G
 y=-1   /// steep slope (unsafe) ///
        x=0   x=1   x=2   x=3   x=4
```

[X] is the boulder at (2,0), detected by the camera while the rover is at (1,0).

Decision steps

Step	Rover decision	Reason

1	Detect boulder at (2,0), straight path blocked	Obstacle percept
2	**Stop**	Safety reflex
3	Reject the route via y = −1	Slope is above the safe limit
4	Plan the route via y = +1	Only safe detour
5	Path: (0,0) → (1,0) → (1,1) → (2,1) → (3,1) → (3,0) → (4,0)	Avoids the obstacle

Cost: the blocked straight route would cost 4 units. The safe route costs **6 units**, which is 2 extra units for safety. No safe route can be shorter than 6, because the shortest possible path (4) is blocked and any detour needs at least 2 extra moves. The rover also checks that battery after the trip ≥ reserve level before moving.

5.6 Pseudocode

```
function MARS_ROVER_AGENT(percept):
  state ← UPDATE_MODEL(state, percept)       // map, position, battery
  if percept.obstacle_distance < STOP_DIST or percept.tilt > TILT_LIMIT:
    return STOP                              // safety reflex
  if state.battery < RESERVE:
    return CHARGE_OR_SLEEP
  if state.at_target:
    return DO_SCIENCE_TASK()                 // image, drill, analyse, store
  path ← A_STAR(state.position, state.goal, avoid = obstacles + unsafe_slopes)
  if path = FAILURE:
    return REQUEST_HELP_FROM_EARTH()
  if state.data_ready and state.link_available:
    return TRANSMIT_DATA
  return NEXT_STEP(path)
```

5.7 Key Capabilities Explained

- **Decision-making:** a goal/utility-based choice that ranks safety first, then energy, then science value.
- **Obstacle avoidance:** the camera and range sensors detect obstacles, the reflex layer stops the rover, and the planner finds a safe detour.

- **Energy efficiency:** it chooses low-cost routes, avoids needless movement, charges when the battery is low, and schedules heavy tasks when power is available.
- **Scientific data collection:** it positions itself at targets, uses the arm and instruments, stores results and transmits them to Earth.

5.8 Why an Autonomous Agent Is Useful on Mars

- **Communication delay:** signals take several minutes (the one-way time changes with planet positions), so live remote control is not practical.
- It must react immediately to dangers such as obstacles, steep slopes and low power.
- Mars is **too dangerous and distant for humans**.
- The rover can keep working between the commands sent from Earth, so more science gets done.
- It handles **unexpected situations** locally, protecting an expensive mission.

5.9 Advantages and Limitations

Advantages	Limitations
Works without continuous human control	Limited computing power and energy
Safer navigation through obstacle avoidance	Sensors can be noisy or blocked by dust
Explores more area and collects more data	Cannot be physically repaired
Saves energy through planning	Unpredictable terrain can cause failures

5.10 Conclusion

The Mars Rover is an autonomous, goal/utility-based agent with a layered architecture: sensors → perception → world model → decision maker → planner and safety reflex → actuators. It puts safety first, conserves energy and collects science data. In the example it detected a boulder, stopped, rejected the unsafe slope and followed a safe 6-unit detour to the goal. Autonomy is essential because Earth-to-Mars delays rule out real-time control.
